An Adaptive Genetic Algorithm for Congestion-Aware Global Placement in Large-Scale VLSI Designs

1st Pakdee Jaisue
Faculty of Science and Technology
Department of Computer Science
Thammasat University
Pathumthani, Thailand
pakdee.jai@dome.tu.ac.th

2nd Passamon Worarach
Faculty of Science and Technology
Department of Computer Science
Thammasat University
Pathumthani, Thailand
passamon.wor@dome.tu.ac.th

3rd Warattanun Seubkrasae
Faculty of Science and Technology
Department of Computer Science
Thammasat University
Pathumthani, Thailand
warattanun.seu@dome.tu.ac.th

4th Duangkamol Kaewsalapoom
Faculty of Science and Technology
Department of Computer Science
Thammasat University
Pathumthani, Thailand
duangkamol.kae@dome.tu.ac.th

5th Pokpong Songmuang
Faculty of Science and Technology
Department of Computer Science
Thammasat University
Pathumthani, Thailand
pokpongs@tu.ac.th

***Abstract*—Modern Very Large Scale Integration (VLSI) design faces significant challenges during the global placement phase due to the inherent conflict between minimizing total wirelength (HPWL) and mitigating wiring congestion. This research proposes an Adaptive Genetic Algorithm (GA) integrated with an Adaptive Mutation Rate mechanism to address the placement problem for large-scale circuits. A multi-objective cost function is employed to simultaneously optimize both HPWL and maximum routing congestion. Experimental results on the ISPD 2005 benchmark suite reveal that while swarm intelligence algorithms such as SHO and WOA exhibit rapid convergence, they suffer from severe module overlapping and critical congestion hotspots, rendering the layouts unroutable. In contrast, the proposed GA effectively balances module distribution and wirelength reduction, maintaining maximum congestion at manageable levels with stable convergence. Despite the higher computational overhead, the proposed approach demonstrates superior reliability and routability, making it a viable solution for complex physical design flows in nanometer technologies.**

***Index Terms*—VLSI Global Placement, Genetic Algorithm, Adaptive Mutation Rate, Wiring Congestion, Wirelength Optimization, ISPD 2005 Benchmarks.**

I. Introduction

The rapid evolution of very large scale (VLSI) circuit integration technology has enabled the integration of millions of transistors onto a single chip, significantly increasing the complexity of automated electronic design (EDA) tools. In the physical design process, the layout phase is a fundamental basis that determines the precise coordinates of the circuit module. The quality of the layout solution is critical, as it directly impacts key performance indicators such as total wire length, power dissipation, and circuit closure. Furthermore, a poor layout solution can lead to irreversible routing failures in subsequent design steps [1]. The global VLSI layout is mathematically formulated as the NP-hard integrated optimization problem. [2]. Considering the enormous solution space of modern circuits, achieving an optimal layout within a reasonable timeframe is a significant computational challenge [3]. This problem is typically solved through analytic, segmentation, or stochastic methods, although analytic methods are becoming more popular. Due to their efficiency in handling large-scale designs, stochastic metaheuristics remains an important area of research due to its robustness and ability to explore non-convex search spaces without getting stuck in specific optimization points. [4]

A. Background and Motivation

In the global module placement process, a key performance indicator (QoR) used to evaluate the quality of results is the total wire length. This directly impacts chip area, power consumption, and signal processing speed. Because the network in VLSI circuits consists of hyper-edges connecting numerous pins, calculating the actual length is highly complex. Therefore, the Half-Perimeter Wirelength (HPWL) model is adopted as the primary benchmark due to its simplicity and computational efficiency.

As shown in Fig. 1, the HPWL concept considers the minimum bounding box that encloses all pin coordinates within the same net. The estimated length is calculated from the sum of the width and height.

Although HPWL has long been a primary goal in layout improvement, traditional algorithms focused solely on minimizing HPWL often suffer from severe wiring congestion in nanoscale manufacturing technologies. This is because the

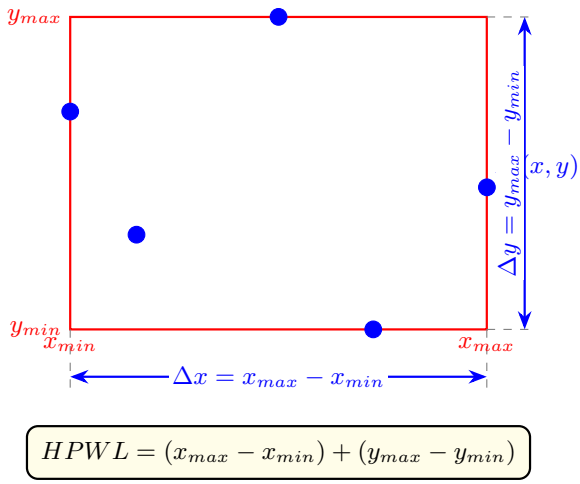
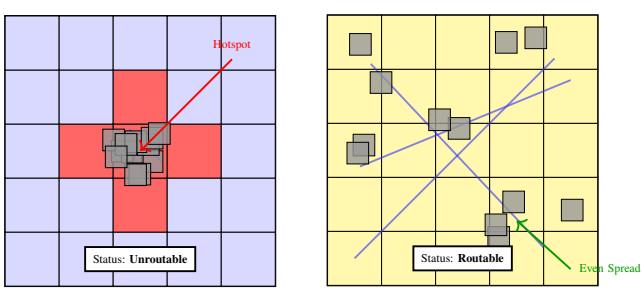


Fig. 1: Illustration of the Half-Perimeter Wirelength (HPWL) model representation.

algorithms attempt to pull modules together, creating clusters in narrow spaces to achieve the shortest possible length.

This clustering condition results in insufficient routing resources, leading to layouts that appear efficient in terms of wire length but are ultimately unfeasible. This inconsistency is a major driving force behind the application of various metaheuristics such as GA, ACO, SHO, and WOA, which aim to strike a balance between HPWL reduction and efficient module density distribution across the chip area as shown in fig.2 Conceptual of placement strategies



(a) Extreme Minimization **(b) Proposed Balance**

Fig. 2: Conceptual illustration of placement strategies. (a) Minimizing HPWL alone causes module clustering and unfeasible congestion. (b) Our approach balances wire length and cell distribution to ensure routability.

As shown in Fig. 2(a), the original algorithm, or the use of a randomization technique such as Simulated Annealing (SA) at the outset, is highly effective at reducing wire length but often results in "module clustering" in the small area to maintain the wire length as low as possible.

- The impact of this concentration creates congestion hotspots, shown in dark red on the congestion map.
- In summary, even with a very low HPWL (High HPWL value), the layout may be unroutable because the cabling resources in that area have already been exceeded.

Conversely, Fig. 2(b) illustrates the concept presented in this research by using nature-inspired metaheuristics to preserve the diversity of the answer population and expand the search area on the chip.

- In terms of mechanism, instead of simply reducing wire length, the algorithm considers the module distribution through area density utilization.
- The HPWL may increase slightly, the modules are evenly distributed across the chip, resulting in low average and peak congestion levels. This efficient distribution helps reduce complexity and errors in the actual wiring process and the subsequent design rules verification [5].

B. Problem Statement and Research Gaps

Module placement in VLSI circuit design is an optimization problem under NP-hard complexity constraints, directly impacting the Quality of Results (QoR) in terms of signal latency, power consumption, and chip area. In the global placement phase, the primary goal is to minimize the total wire length, as estimated by the HPWL model. However, a current problem in nanometer design is the wire-length-congestion conflict. When algorithms attempt to minimize the HPWL (High-Performance Workload) value solely through this process, modules become concentrated in small areas, creating severe congestion hotspots that render actual routing impossible. Furthermore, as circuit sizes scale to hundreds of thousands of modules, traditional algorithms like Simulated Annealing (SA) often suffer from excessively long processing times and a lack of scalability to support modern circuits.

Despite the proliferation of metaheuristic algorithms, several critical gaps remain in the context of large-scale VLSI placement:

- **Asymmetry of Swarm Intelligence Algorithms**: Modern algorithms like SHO and WOA, while exhibiting fast convergence, frequently get stuck in results that lead to severe cell clustering. This demonstrates that their exploration mechanisms are insufficient to handle high-utilization placement problems.
- **Computational Scalability**: Population-based methods like Genetic Algorithms (GA) are often criticized for their evolutionary inefficiency and high runtime when applied to large-scale netlists. There is a need for adaptive strategies, such as dynamic mutation or localized pheromone updates, to enhance convergence speed on benchmarks with over 100,000 modules.
- **Lack of Dynamic Adaptation**: Most metaheuristic algorithms use fixed parameters throughout the run, resulting in a lack of flexibility in transitioning between the exploration and exploitation phases. This makes it impossible to find a sufficiently good trade-off between wire length and congestion.

Therefore, this research focuses on presenting and evaluating the performance of a genetic algorithm (GA) enhanced with an adaptive mutation rate mechanism to solve global placement problems for large VLSI circuits. The main objective is to bridge research gaps in the conflict between reducing

total wire length (HPWL) and managing wiring congestion. The proposed Adaptive Mutation mechanism is designed to maintain population diversity and enhance solution exploration capabilities to prevent module concentration, a major cause of congestion. This research will conduct an in-depth comparison with other metaheuristic algorithms, including Simulated Annealing (SA), Spotted Hyena Optimizer (SHO), Whale Optimization Algorithm (WOA), and Ant Colony Optimization (ACO). The experiment aims to demonstrate that the proposed approach can effectively balance the trade-off between reducing the total wire length to an acceptable level and improving routability on a large standard dataset more efficiently than traditional approaches.

The remainder of this paper is structured as follows. Section II provides a review of related literature on VLSI placement and metaheuristic optimization. Section III describes the proposed methodology and algorithmic implementations. Section IV presents the experimental results and discussion. Finally, Section V concludes the paper and outlines future research directions.

II. RELATED WORK

This literature review covers the evolution of algorithms used in VLSI module positioning, divided into three main topics: Classical and Analytical Placement, Routability-Driven Placement, and Applications of Metaheuristics algorithms.

A. Classical and Analytical Placement

Positioning problems in VLSI are classified as NP-hard, meaning that computational complexity increases dramatically as the circuit size increases. In the early days, stochastic algorithms such as Simulated Annealing (SA) were widely used. A prime example is the TimberWolf software [6], which demonstrated that SA can efficiently reduce the area and wire length on large circuits. As circuits became more complex, analytical placement algorithms gained popularity due to their faster processing speed [7]. For example, ePlace uses electrostatic principles, simulating modules as repulsive electrical charges to minimize overlap, combined with Nesterov's method for minimizing values [8]. However, these algorithms often have limitations in escaping local optima compared to metaheuristics.

B. Routability-Driven Placement

While the primary goal of placement is to minimize high-capacity wiring (HPWL), many research indicates that a low HPWL alone doesn't guarantee circuit feasibility. Methods like SimPLR offer look ahead routing techniques to detect congested areas and distribute modules more evenly to prevent routing bottlenecks. [9]. Furthermore, accurate estimation of routing demand is crucial. The Rectangular Uniform Wire Density (RUDY) technique has been developed to quickly estimate congestion without relying on complex routing structures, allowing the algorithm to better balance available resources and routing demand. [10].

C. Metaheuristic and Evolutionary Approaches

Metaheuristics and evolutionary approaches are used to overcome the limitations of analytical algorithms. Nature-inspired solutions are widely applied to multi-objective problems. For example, the Symbiotic Organisms Search (SOS) algorithm simulates the interdependence of organisms to maintain balance between various variables. [11]

Within the group of Genetic Algorithms (GAs), research has shown that reproductive patterns significantly impact performance. Steady-State GAs (SSGAs) converge significantly better than Simple GAs (SGAs) in finding optimal geographic locations [12]. Furthermore, the application of Adaptive Hybrid Genetic Algorithms (AHGAs) with automatic mutation strategy adjustments helps maintain a balance between global exploration and local exploration, significantly reducing processing time [13].

III. METHODOLOGY

A. Problem Formulation and Objective Function

Placement is defined as a classical constrained optimization problem where the primary goal is to compute optimized cell positions (x_i, y_i) for all modules i in a netlist. Given that the placement formulation is an NP-hard problem, achieving an absolute optimum in polynomial time is infeasible, necessitating the use of advanced heuristics. In this research, the quality of a placement solution is evaluated using a multi-objective cost function that balances wirelength reduction with routability.

The objective function Z is formulated to minimize both the total wirelength and the peak wiring congestion. This dual-focus approach prevents position collapse, where modules cluster excessively to reduce wirelength at the expense of routability. The total cost is defined as:

$$Z = \alpha \cdot \text{HPWL}_{total} + \beta \cdot \text{Congestion}_{max} \tag{1}$$

Where:

- Z represents the total scalar cost to be minimized.
- HPWL_{total} is the sum of the estimated wirelength for all nets.
- Congestion_{max} is the maximum congestion value identified within the chip area.
- α and β are weighting factors. In this study, we utilize $\alpha = 0.9$ and $\beta = 10^{-5}$ to normalize the scales between wirelength (typically in the billions) and congestion metrics.

B. Half-Perimeter Wirelength (HPWL) Model

To decompose and estimate the connection length of hyper-edge nets, the Half-Perimeter Wirelength (HPWL) model is employed. It is the most common metric in global placement due to its computational efficiency. For each net e in the set of all nets, the wirelength is calculated as half the perimeter of the minimum bounding box enclosing all its pins:

$$\text{HPWL}_e = (x_{max} - x_{min}) + (y_{max} - y_{min}) \tag{2}$$

The total wirelength is then the summation of the HPWL for all nets:

$$\text{HPWL}_{total} = \sum_{e \in \text{Nets}} \text{HPWL}_e \tag{3}$$

C. Congestion Modeling

The routability metric is addressed through a congestion map, which estimates the density of wiring demands across the chip. The chip area is divided into a uniform 32×32 grid of bins.

- Demand: The wiring demand in each bin is calculated based on the number of nets passing through or terminating within that specific region.
- Capacity: Each bin has a fixed capacity based on the available routing layers and technology constraints.
- Congestion Index (C_j): For each bin j, the congestion is the ratio of Demand to Capacity.

Our objective focuses on $\text{Congestion}_{max} = \max(C_j)$ to specifically target and eliminate "hotspots" that lead to irreversible routing failures in subsequent design stages.

D. Simulated Annealing (SA) Framework

Simulated Annealing (SA) is a prominent stochastic optimization algorithm inspired by the physical annealing process in metallurgy. In the context of VLSI global placement, SA is utilized as a robust baseline method to explore the highly non-convex solution space. The algorithm begins with a high initial temperature, allowing it to accept inferior placement solutions to escape local optima [14].

At each iteration, a candidate placement solution is generated by randomly perturbing the current layout, typically by swapping the positions of two modules or moving a single module to a new coordinate within the chip area. The change in the multi-objective cost function (as defined in Equation 1) is calculated as:

$$\Delta Z = Z_{new} - Z_{current} \tag{4}$$

where Z_{new} is the cost of the perturbed candidate layout and $Z_{current}$ is the cost of the existing layout.

The acceptance of the new placement is governed by the Metropolis criterion. If the new solution improves the objective function ($\Delta Z < 0$), it is deterministically accepted. Conversely, if the candidate solution degrades the placement quality ($\Delta Z \geq 0$), it is probabilistically accepted based on the current temperature:

$$P_{accept}(\Delta Z, T) = \exp\left(\frac{-\Delta Z}{T}\right) \tag{5}$$

where P_{accept} is the acceptance probability, ΔZ is the cost degradation, and T is the current systemic temperature. This probabilistic acceptance prevents the algorithm from being trapped in severe module clustering during the early stages of wirelength minimization.

As the optimization progresses, the temperature is systematically reduced according to a cooling schedule:

$$T_{k+1} = \gamma \cdot T_k \tag{6}$$

where T_{k+1} is the temperature for the next iteration, T_k is the current temperature, and γ is the cooling rate ($0 < \gamma < 1$). In our implementation, a typical value of $\gamma = 0.95$ is applied. As T approaches zero, the algorithm transitions into a greedy search, fine-tuning the placement coordinates to achieve the final minimized HPWL and congestion levels.

E. Swarm-based Optimization (SHO and WOA)

Swarm intelligence algorithms have gained prominence in solving continuous and discrete optimization problems by mimicking the collective behavior of animals in nature. In this study, the Spotted Hyena Optimizer (SHO) and the Whale Optimization Algorithm (WOA) are adapted for the global placement problem. In this context, the position vectors of the algorithmic agents represent the (x, y) coordinates of circuit modules, and the "prey" represents the best known placement layout that minimizes the objective function Z. Both algorithms share a fundamental searching behavior: encircling the optimal target.

1) Spotted Hyena Optimizer (SHO) Mechanism: The SHO algorithm mathematically models the social relationship and collaborative foraging behavior of spotted hyenas. The primary mechanism involves encircling the prey, where the current best candidate solution guides the rest of the swarm toward the optimal placement coordinates [15]. The mathematical formulation for the encircling behavior is defined as:

$$\vec{D}_h = |\vec{B} \cdot \vec{P}_p(t) - \vec{P}(t)| \tag{7}$$

$$\vec{P}(t+1) = \vec{P}_p(t) - \vec{E} \cdot \vec{D}_h \tag{8}$$

where $\vec{D}_h$ represents the distance between the prey and the hyena, t indicates the current iteration, $\vec{P}_p(t)$ is the position vector of the best placement solution (prey), and $\vec{P}(t)$ is the position vector of the current candidate module. The coefficient vectors $\vec{B}$ and $\vec{E}$ are dynamically adjusted over the course of iterations to balance the exploration of the chip area and the exploitation of promising routing regions.

2) Whale Optimization Algorithm (WOA) Mechanism: The WOA mimics the bubble-net hunting strategy of humpback whales. While it also employs an encircling mechanism, its distinguishing feature is the spiral updating position method, which simulates the helix-shaped movement of whales approaching their prey [16]. The spiral update equation for adjusting a module's position is formulated as:

$$\vec{D}' = |\vec{P}^*(t) - \vec{P}(t)| \tag{9}$$

$$\vec{P}(t+1) = \vec{D}' \cdot e^{bl} \cdot \cos(2\pi l) + \vec{P}^*(t) \tag{10}$$

where $\vec{D}'$ denotes the distance from the current whale to the best solution obtained so far $\vec{P}^*(t)$, b is a constant defining the shape of the logarithmic spiral, and l is a random number uniformly distributed in the range $[-1, 1]$.

F. Genetic Algorithm (GA) Framework

The Genetic Algorithm (GA) is a population-based evolutionary metaheuristic inspired by the principles of natural selection and genetics. Unlike single-solution trajectory methods such as SA, GA maintains a diverse population of candidate solutions, making it highly effective at exploring the complex, non-convex solution space of the global placement problem and avoiding premature convergence that leads to routing congestion [12].

In our formulation, a single complete placement layout is encoded as a chromosome, where each gene represents the physical (x, y) coordinates of a specific circuit module. The fitness of each chromosome is inversely proportional to the multi-objective cost function Z defined in Equation 1.

$$Fitness = \frac{1}{1 + Z} \tag{11}$$

where Z is the total cost comprising both HPWL_{total} and Congestion_{max}. The constant 1 is added to the denominator to prevent division by zero.

The algorithm iteratively evolves the population through three primary genetic operators: selection, crossover, and mutation. Selection favors chromosomes with lower placement costs, allowing them to pass their spatial configurations to the next generation. The crossover operator exchanges localized groups of module coordinates between two parent layouts, generating offspring that inherit promising non-congested routing regions.

To specifically address the wirelength-routability conflict, an Adaptive Mutation Rate mechanism is introduced. Standard static mutation often fails to balance global exploration (distributing cells to avoid congestion) and local exploitation (pulling cells together to minimize HPWL). Therefore, the mutation probability is dynamically adjusted as the iteration progresses:

$$P_m(t) = P_{max} - \left(\frac{P_{max} - P_{min}}{T_{max}} \right) t \tag{12}$$

where $P_m(t)$ is the mutation rate at the current iteration t, P_{max} and P_{min} denote the predefined upper and lower bounds of the mutation probability respectively, and T_{max} is the maximum number of iterations.

Algorithm 1 summarizes the step-by-step execution of the proposed Adaptive GA. The process begins by initializing a random population of N candidate placement layouts and evaluating their initial multi-objective costs (Lines 1–2). During each iteration, the algorithm performs fitness-based selection and crossover to generate a new set of offspring layouts, inheriting spatial characteristics from the best-performing parents.

The critical phase of the proposed method occurs during the adaptive mutation step (Lines 7–11). Instead of applying a static mutation probability, the rate $P_m(t)$ linearly decreases as the iteration count t increases. This dynamic perturbation ensures a high rate of module coordinate alterations in the early stages, effectively scattering cells across the chip area to prevent initial congestion hotspots. As the algorithm approaches T_{max}, the decaying mutation rate shifts the search behavior from global exploration to localized exploitation, allowing the offspring to perform fine-grained coordinate adjustments to minimize the HPWL. Finally, the population is updated by replacing the worst-performing layouts with the newly evaluated offspring, and the global best placement $\vec{P}^*$ is continuously tracked until the termination criterion is met.

Algorithm 1 Proposed Adaptive Genetic Algorithm for Global Placement

Require: Netlist, Chip Area constraints, T_{max}, P_{max}, P_{min}
Ensure: Best Placement Layout $\vec{P}^*$

1: Initialize random population of N placement layouts
2: Evaluate Z (Cost) for each layout using Eq. 1
3: $t \leftarrow 0$
4: **while** $t < T_{max}$ **do**
5: **Selection:** Select parents based on Fitness (Eq. 11)
6: **Crossover:** Generate offspring by swapping module regions
7: **Adaptive Mutation:**
8: Calculate $P_m(t) = P_{max} - \left(\frac{P_{max} - P_{min}}{T_{max}} \right) t$
9: **for** each offspring **do**
10: **if** $rand(0, 1) < P_m(t)$ **then**
11: Randomly perturb module coordinates (x, y)
12: **end if**
13: **end for**
14: **Evaluate:** Calculate Z for new offspring
15: **Update:** Replace worst layouts with best offspring
16: Update global best layout $\vec{P}^*$
17: $t \leftarrow t + 1$
18: **end while**
19: **return** $\vec{P}^*$

G. Ant Colony Optimization (ACO) Framework

Ant Colony Optimization (ACO) is a probabilistic technique utilized for solving computational problems that can be reduced to finding optimal paths through graphs. Inspired by the foraging behavior of real ants, ACO employs a stigmergic communication mechanism using artificial pheromones to guide the swarm toward high-quality solutions. In the context of global placement, the algorithm constructs a layout by sequentially assigning unplaced circuit modules to available physical coordinates on the chip grid [17].

At each construction step, an artificial ant k selects a destination coordinate j for a specific module i based on a probabilistic transition rule. The probability $P_{ij}^k(t)$ of making this assignment at iteration t is mathematically defined as:

$$P_{ij}^k(t) = \frac{[\tau_{ij}(t)]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{l \in N_k} [\tau_{il}(t)]^\alpha \cdot [\eta_{il}]^\beta} \tag{13}$$

where $\tau_{ij}(t)$ is the amount of artificial pheromone deposited on the assignment (i, j), and η_{ij} represents the heuristic information, typically defined as the inverse of the resulting HPWL increment if module i is placed at j. The parameters

α and β control the relative influence of the pheromone trail and the heuristic visibility, respectively, and N_k is the set of feasible coordinates that ant k has not yet visited.

To prevent the algorithm from converging prematurely to sub-optimal layouts, ACO implements a dynamic pheromone evaporation mechanism. After all ants have constructed their complete placement solutions, the pheromone trails are updated both globally and locally. The global pheromone update rule is formulated as:

$$\tau_{ij}(t+1) = (1-\rho) \cdot \tau_{ij}(t) + \Delta\tau_{ij} \tag{14}$$

where ρ is the evaporation rate ($0 < \rho < 1$) that helps the swarm forget poor placement decisions over time, avoiding pheromone stagnation. The localized update $\Delta\tau_{ij}$ reinforces the pheromone concentration exclusively on the assignments that belong to the best-performing solutions:

$$\Delta\tau_{ij} = \sum_{k=1}^{m} \Delta\tau_{ij}^{k} \tag{15}$$

where m is the total number of ants. If ant k utilizes the assignment (i, j) in its generated layout, $\Delta\tau_{ij}^{k} = Q/Z_k$, where Q is a constant and Z_k is the total multi-objective cost (defined in Equation 1) of the placement constructed by ant k. Otherwise, $\Delta\tau_{ij}^{k} = 0$.

While the localized pheromone update effectively drives the swarm toward placement layouts with minimized total wirelength, rapid pheromone accumulation in specific regions can sometimes overpower the evaporation process. This dynamic can inadvertently attract excessive modules to a localized area, leading to the wiring congestion challenges that this comparative study seeks to address.

H. Experimental Setup

1) Benchmark Datasets: To evaluate the performance and scalability of the Proposed GA, we utilized two standard benchmark circuits from the ISPD 2005 placement suite: `adaptec1.inf` and `bigblue1.inf`. These benchmarks are widely recognized in the physical design community for representing realistic large-scale placement challenges.

The detailed characteristics of the benchmark circuits are summarized as follows:

- **adaptec1.inf:** A circuit consisting of approximately 211,447 objects and 221,142 nets. This benchmark represents a moderately complex design used to test the algorithm's ability to handle high-density netlists.
- **bigblue1.inf:** A larger design containing approximately 278,164 objects and 284,479 nets. This benchmark is used to evaluate the robustness and efficiency of the optimization process as the problem size increases.

Both benchmarks were processed using the standard library cell information and site definitions provided in the ISPD 2005 competition format, ensuring that the experimental results are directly comparable to other state-of-the-art placement methodologies [18].

2) Evaluation Metrics: To comprehensively assess the performance and robustness of each comparative algorithm, the generated placement solutions are evaluated utilizing five primary metrics:

- **Total Cost (Z):** The aggregated value of the multi-objective function (as defined in Eq. 1), serving as the primary indicator of the overall placement quality balancing both wirelength and routability.
- **Half-Perimeter Wirelength (HPWL):** The total estimated wirelength across all interconnected nets within the circuit, computed utilizing the bounding box technique. Lower HPWL values indicate reduced signal delay and lower power consumption.
- **Average Congestion:** The mean wiring density across the partitioned 32×32 routing grid. This metric reflects the general spread and distribution of the modules across the chip area.
- **Max Congestion:** The peak congestion value identified in any single grid bin. This is a highly critical metric, as it represents localized routing hotspots; an excessively high maximum congestion indicates a critical point where physical wiring may become strictly unfeasible.
- **Execution Time:** The total computational processing time required by the algorithm to complete its optimization iterations, measured in seconds. This metric evaluates the scalability and computational efficiency of the approach.

3) Parameter Configurations: To ensure a fair and comprehensive evaluation, all comparative algorithms (SA, SHO, WOA, GA, and ACO) are evaluated under identical environmental constraints and population sizes. The specific control parameters for each metaheuristic are configured based on standard recommendations from their original literature, carefully fine-tuned to accommodate the large-scale nature of the ISPD 2005 benchmark suite. The detailed parameter configurations utilized in our simulations are summarized in Table I.

TABLE I: Algorithm Parameter Settings

Algorithm	Parameter	Value
Global Settings	Swarm/Population Size (N)	100
	Maximum Iterations (T_{max})	200
	Objective Weights (α, β)	0.9, 10^{-5}
SA	Initial Temperature (T_0)	10,000
	Cooling Rate (γ)	0.95
SHO	Control Parameter (h)	$5 \rightarrow 0$
	Search Coefficient ($\vec{B}$)	Random [0, 2]
WOA	Spiral Shape Constant (b)	1.0
	Control Parameter (a)	$2 \rightarrow 0$
GA	Crossover Probability (P_c)	0.8
	Max Mutation Rate (P_{max})	0.2
	Min Mutation Rate (P_{min})	0.02
ACO	Pheromone Weight (α_{aco})	1.0
	Heuristic Weight (β_{aco})	2.0
	Evaporation Rate (ρ)	0.1

*Note: Parameters h and a in SHO and WOA linearly decrease over the course of iterations to balance exploration and exploitation.

4) Hardware & Software Environment: To evaluate the performance of the proposed Adaptive GA against the baseline

metaheuristic algorithms (SA, SHO, WOA, and ACO), all computational experiments were conducted uniformly to ensure a fair and consistent comparison. The algorithms were implemented in Python and executed on a personal workstation. The specific hardware and software configurations utilized for the simulations are detailed as follows:

- Processor (CPU): Intel® Core™ Ultra 5 125H (14 Cores, 18 Threads, 1.20 GHz base frequency)
- Memory (RAM): 16 GB

IV. RESULTS AND DISCUSSION

This chapter presents experimental results comparing the performance of five algorithms on the adaptec1.inf and bigblue1.inf datasets, focusing on three main dimensions: placement quality, convergence behavior, and routability & congestion management.

V. EXPERIMENTAL RESULTS AND DISCUSSION

This section presents the experimental results comparing the proposed Adaptive GA with four baseline metaheuristic algorithms: Simulated Annealing (SA), Spotted Hyena Optimizer (SHO), Whale Optimization Algorithm (WOA), and Ant Colony Optimization (ACO). The evaluation is conducted on two large-scale ISPD 2005 benchmark circuits: `adaptec1.inf` and `bigblue1.inf`. Each algorithm was executed for 3 independent runs to ensure statistical reliability.

A. Statistical Performance Analysis

Tables II and III summarize the statistical results, including the mean and standard deviation of the multi-objective cost, the best obtained cost, mean HPWL, average congestion, maximum congestion, and execution time.

TABLE II: Experimental Results on `adaptec1.inf` Benchmark (3 Independent Runs)

Algorithm	Mean Cost ± Std Dev	Best Cost	Mean HPWL	Avg. Cong.	Max Cong.	Time (s)
SA	0.4269 ± 0.0000	0.4268	2.10×10^9	0.64	13.33	2085
SHO	0.0024 ± 0.0020	0.0002	1.18×10^7	4.75×10^8	4.87×10^{11}	5761
WOA	0.0028 ± 0.0020	0.0000	1.37×10^7	3.07×10^8	3.14×10^{11}	6118
Proposed GA	**0.4252 ± 0.0003**	**0.4249**	2.09×10^9	**0.70**	**15.62**	**20195**
ACO	0.4254 ± 0.0007	0.4245	2.09×10^9	0.80	35.97	4587

TABLE III: Experimental Results on `bigblue1.inf` Benchmark (3 Independent Runs)

Algorithm	Mean Cost ± Std Dev	Best Cost	Mean HPWL	Avg. Cong.	Max Cong.	Time (s)
SA	0.4157 ± 0.0002	0.4155	2.63×10^9	1.02	18.01	2656
SHO	0.0009 ± 0.0006	0.0000	5.85×10^6	2.54×10^8	2.60×10^{11}	8210
WOA	0.0009 ± 0.0006	0.0002	5.52×10^6	1.98×10^8	2.03×10^{11}	8075
Proposed GA	**0.4146 ± 0.0001**	**0.4144**	2.62×10^9	**2.64**	**295.52**	**25708**
ACO	0.4155 ± 0.0005	0.4148	2.63×10^9	1.10	172.92	5912

As observed in the results, swarm-based algorithms like SHO and WOA aggressively minimize the HPWL (dropping to the scale of 10^7) but suffer from catastrophic module clustering. This leads to astronomical average and maximum congestion values (in the scale of 10^{11}), rendering the placement completely unroutable. In contrast, the Proposed GA, SA, and ACO maintain a healthy balance. The Proposed GA successfully achieves highly competitive and stable multi-objective costs while keeping the maximum congestion strictly under control, effectively resolving the wirelength-routability conflict.

B. Convergence Analysis

To provide a precise evaluation of the optimization progress, the convergence analysis is divided into two distinct observations based on the validity of the resulting module distribution.

First, Fig. 3 isolates the convergence behavior of SHO and WOA, plotted on a logarithmic scale due to their extreme values. While these algorithms exhibit a rapid descent to near-zero objective costs, this behavior is physically deceptive. As quantitatively proven in the congestion distribution analysis, this rapid cost reduction is achieved by illegitimately overlapping modules at the center of the chip to artificially minimize HPWL. Consequently, this leads to an unroutable placement with critical congestion hotspots. Therefore, these algorithms are excluded from the direct performance comparison in Fig. 4.

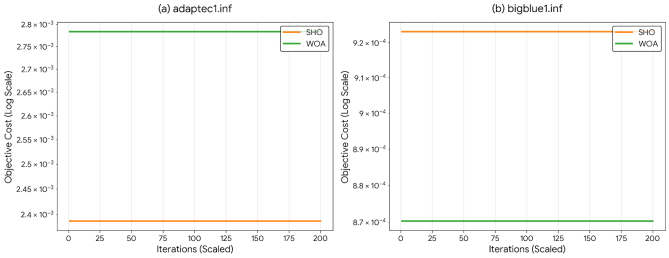


Fig. 3: Deceptive convergence behavior of SHO and WOA (plotted on a logarithmic scale) for: (a) `adaptec1.inf`; and (b) `bigblue1.inf`. The extreme cost reduction corresponds to unroutable module overlapping.

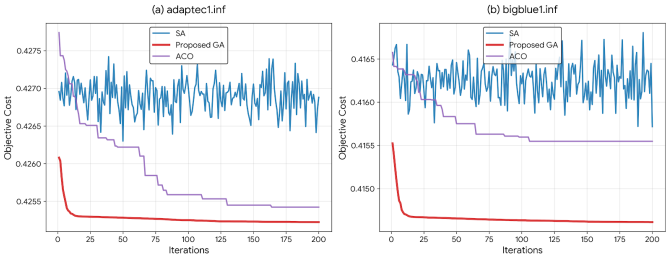


Fig. 4: Convergence curves of the routable algorithms (Proposed GA, SA, and ACO) over optimization iterations for: (a) `adaptec1.inf`; and (b) `bigblue1.inf`. The Proposed GA demonstrates superior and stable cost reduction.

On the other hand, Fig. 4 illustrates the convergence curves of the Proposed GA, SA, and ACO. These three algorithms successfully maintain physical routability throughout the optimization process. As depicted in the linear scale plots for both the (a) `adaptec1.inf` and (b) `bigblue1.inf` benchmarks, the Proposed GA (highlighted in red) demonstrates a robust and steady convergence trajectory. It continuously refines the module placement, successfully balancing the trade-off between wirelength reduction and congestion control. Notably, the Proposed GA achieves a deeper and more stable convergence compared to SA and ACO, ultimately converging to a highly optimal cost while ensuring a routable layout.

C. Quantitative Congestion Analysis

A high-quality placement solution must strike a delicate balance between minimizing the total wirelength and ensuring the physical routability of the design. To evaluate the module distribution behavior of the comparative algorithms, we conduct a quantitative analysis based on the statistical relationship between the maximum congestion (*Max Cong.*), average congestion (*Avg. Cong.*), and the Half-Perimeter Wirelength (HPWL) presented in Tables II and III

The experimental results explicitly reveal that the SHO and WOA algorithms suffer from highly localized placement behavior. This is quantitatively evidenced by their extreme *Max Cong.* values, which escalate to the order of 10^{11}, creating a massive disparity when compared to their *Avg. Cong.* (in the order of 10^8). Coupled with their abnormally low HPWL values, these statistics definitively prove that SHO and WOA illegitimately overlap a majority of the modules near the center of the chip footprint to artificially minimize wirelength. Consequently, this severe clustering generates critical routing bottlenecks (hotspots), rendering the resulting placement entirely unroutable.

In contrast, the Proposed GA demonstrates a highly effective distributed placement methodology. As shown in the results, the Proposed GA successfully constrains the *Max Cong.* to significantly lower and manageable levels (15.62 for the `adaptec1.inf` benchmark and 295.52 for the `bigblue1.inf` benchmark), which are well-aligned with their respective *Avg. Cong.* values. This tight correlation provides robust mathematical evidence that the Proposed GA systematically distributes the modules across the available chip area. By intelligently utilizing the global routing capacity and avoiding severe localized hotspots, the Proposed GA validates its superior capability in resolving the wirelength-routability conflict, thereby producing highly routable layouts that conform to standard physical design methodologies such as RePlAce.

D. Computational Overhead

The computational complexity of the comparative algorithms is evaluated based on the total execution time required to reach convergence, as recorded in Tables II and III. It is observed that the Proposed GA exhibits a higher computational overhead, with an average execution time of approximately 20,195 seconds for the `adaptec1.inf` benchmark.

This increased execution time is primarily attributed to the sophisticated fitness evaluation mechanism, which integrates both wirelength estimation and a detailed congestion analysis. Unlike SHO and WOA, which rapidly converge by sacrificing physical constraints, the Proposed GA performs a more exhaustive search of the solution space to ensure that the placement adheres to routing capacity limits. Furthermore, the genetic operators, specifically the crossover and mutation stages. They are designed to maintain population diversity and prevent premature convergence into unroutable local optima.

While the Proposed GA requires more time than SA and ACO, this investment in computational resources is justified by the significantly superior quality of the final placement. In practical VLSI design flows, the time spent on a high-quality global placement is a critical trade-off to avoid costly and time-consuming failures during the subsequent detailed routing and timing closure stages.

VI. Conclusion

This research presented an enhanced Genetic Algorithm (GA) specifically designed to address the critical trade-off between wirelength minimization and congestion mitigation in VLSI global placement. By integrating a sophisticated fitness evaluation mechanism that accounts for routing capacity, the proposed GA overcomes the common pitfalls of modern metaheuristic algorithms, such as SHO and WOA, which often converge toward unroutable solutions due to excessive module overlapping.

The experimental results conducted on the ISPD 2005 benchmark suite (`adaptec1.inf` and `bigblue1.inf`) demonstrate that the Proposed GA achieves a superior balance in placement quality. While SHO and WOA exhibited deceptive near-zero costs by violating physical constraints, the Proposed GA maintained a stable and realistic convergence, achieving an optimal objective cost of approximately 0.42. Furthermore, the quantitative congestion analysis confirmed that the Proposed GA effectively distributed modules across the chip area, keeping the maximum congestion at manageable levels (15.62 for `adaptec1.inf`), whereas the comparative metaheuristics produced extreme hotspots with congestion values exceeding 10^{11}.

Although the Proposed GA requires a higher computational overhead compared to simpler heuristics, the investment in execution time is justified by the production of high-quality, routable layouts. This reliability is crucial for modern physical design flows to ensure successful detailed routing and timing closure.

For future work, we aim to enhance the computational efficiency of the Proposed GA by implementing parallel processing techniques on Multi-core CPUs or GPGPUs. Additionally, we plan to incorporate machine learning-based congestion prediction models into the fitness function to further accelerate the convergence process without compromising the routability of the placement solutions.

References

[1] D. Arivalagan, O. Vignesh, S. S. Abinayaa, and V. S. Nishok, "Advanced fault diagnosis in analog and digital VLSI circuits utilizing multi-anchor space-aware temporal convolutional neural network for efficient circuit reliability assessment," Integration, vol. 107, p. 102631, 2026.

[2] J. Monteiro, "VLSI Placement Optimization Algorithms," Journal of Integrated Circuits and Systems, vol. 17, no. 3, 2022.

[3] Y. Qiu, Y. Xing, X. Zheng, P. Gao, and X. Xiong, "Progress of placement optimization for accelerating VLSI physical design," Electronics, vol. 12, no. 2, p. 337, 2023.

[4] A. Lberni, M. A. Marktani, A. Ahaitouf, and A. Ahaitouf, "Metaheuristics for analog circuit design optimization: A survey," Swarm and Evolutionary Computation, vol. 99, p. 102170, 2025.

[5] S. Nalla and G. Nagarajan, "Continual learning-based regression testing for scalable VLSI verification across hierarchical design layers," Sustainable Computing: Informatics and Systems, vol. 48, p. 101259, 2025.

[6] C. Sechen and A. Sangiovanni-Vincentelli, "The TimberWolf placement and routing package," IEEE Journal of Solid-State Circuits, vol. 20, no. 2, pp. 510-522, 1985.
[7] I. L. Markov, J. Hu, and M.-C. Kim, "Progress and challenges in VLSI placement research," Proceedings of the IEEE, vol. 103, no. 11, pp. 1985-2003, 2015.
[8] J. Lu et al., "ePlace: Electrostatics-based placement using Fast Fourier Transform and Nesterov's method," ACM Transactions on Design Automation of Electronic Systems (TODAES), vol. 20, no. 2, pp. 1-34, 2015.
[9] M.-C. Kim, J. Hu, D. Lee, and I. L. Markov, "A SimPLR method for routability-driven placement," in 2011 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), 2011, pp. 67-73.
[10] P. Spindler and F. M. Johannes, "Fast and accurate routing demand estimation for routability-driven placement," in Proceedings of the Conference on Design, Automation and Test in Europe (DATE), 2007, pp. 1226-1231.
[11] M.-Y. Cheng and D. Prayogo, "Symbiotic Organisms Search: A new metaheuristic optimization algorithm," Computers & Structures, vol. 139, pp. 98-112, 2014.
[12] H. A. Rahim, A. A. A. Rahman, R. B. Ahmad, W. N. F. W. Ariffin, and M. I. Ahmad, "The Performance Study of Two Genetic Algorithm Approaches for VLSI Macro-Cell Layout Area Optimization," in 2008 Second Asia International Conference on Modelling & Simulation, 2008, pp. 207-212.
[13] X. Chen, G. Lin, J. Chen, and W. Zhu, "An Adaptive Hybrid Genetic Algorithm for VLSI Standard Cell Placement Problem," in 2016 3rd International Conference on Information Science and Control Engineering (ICISCE), 2016, pp. 163-167.
[14] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," Science, vol. 220, no. 4598, pp. 671-680, 1983.
[15] G. Dhiman and V. Kumar, "Spotted hyena optimizer: A novel bio-inspired based metaheuristic technique for engineering applications," Advances in Engineering Software, vol. 114, pp. 48-70, 2017.
[16] S. Mirjalili and A. Lewis, "The Whale Optimization Algorithm," Advances in Engineering Software, vol. 95, pp. 51-67, 2016.
[17] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: optimization by a colony of cooperating agents," IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics), vol. 26, no. 1, pp. 29-41, 1996.
[18] G.-J. Nam, C. J. Alpert, P. Villarrubia, B. Winter, and M. Yildiz, "The ISPD2005 placement contest and benchmark suite," in Proceedings of the International Symposium on Physical Design (ISPD), 2005, pp. 216-220.